

Direct Runtime Compilation of Compositional GSLTs

Metamath and MM0 as didactic guests for PeTTa and ρ

Zarathustra Goertzel Oruži

August 2026 — integration draft

Abstract

We describe a compiler architecture in which an authored generalized syntax–semantics language theory (GSLT) is composed once and specialized into a readable target program, while concrete documents remain runtime data. The distinction is elementary but decisive: a Metamath database or MM0 file must be parsed and checked by a reusable generated verifier; it must not be translated into thousands of target equations and thereby become part of the verifier itself. Syntax declarations encountered in a document update a live parsing environment through ordinary semantic transitions.

The first target is PeTTa, expressed as direct equations, pattern matches, recursion, and explicit result/state terms. A second lowering targets the ρ -calculus directly through processes, channels, and matching structure. Neither route factors through Horn clauses, SLD resolution, Datalog, or a universal worklist machine. Such interpretations may remain useful as independent diagnostic oracles, but they are not the meaning of the source GSLT and are not on the production path. An optional execution IR is allowed where it is earned by multiple targets, not assumed in advance.

The current implementation qualifies bounded direct PeTTa and ρ artifacts for Metamath transition/proof slices and for the MM0 textual pipeline through assertion elaboration. The complete direct Metamath-to- ρ route, complete MM0 definitions, textual proof execution, binary `.mmb` verification, and the full compiler correspondence proofs remain open. We report the construction and the residue separately so that tests are evidence for a compiler whose correctness is structural, rather than a translation-validation shell around an independently invented engine.

1 The compilation question

Suppose a language package L contains signatures Σ_L , equations E_L , rewrites R_L , and an authored composition graph. A concrete document d is written in the language described by L . For a target presentation T , the intended staging is

$$P_T = \text{compile}_T(L), \quad \text{answer}_T(L, d) = \text{run}_T(P_T, \text{parse}_L(d)).$$

The target program P_T depends on L and T , not on d . This is the ordinary compiler/interpreter separation studied by partial evaluation [3, 4]: specialize the language machinery to a language definition, leaving the program input dynamic.

The tempting wrong staging is

$$P_{T,d} = \text{compile}_T(L, d).$$

It can appear to work on small fixtures because every parsed fact is rendered as a target clause. On a real database it duplicates the document, inflates startup cost, and makes a verifier change

whenever its input changes. More seriously, it confuses two semantic roles: a declaration is data consumed by the source fold, whereas a rewrite is program structure defining that fold.

Principle 1 (Runtime-input independence). *For fixed authored package L , target T , and compiler version, compilation is byte-deterministic and independent of the runtime document:*

$$\text{compile}_T(L; d_1) = \text{compile}_T(L; d_2) = \text{compile}_T(L).$$

Document identity may appear in a runtime receipt, but not in the generated program's rule inventory.

This principle is both conceptual and operational. It prevents a database with n declarations from manufacturing $O(n)$ target equations before the checker can begin, and it makes one generated artifact reusable across the whole corpus.

2 GSLTs and composition-preserving lowering

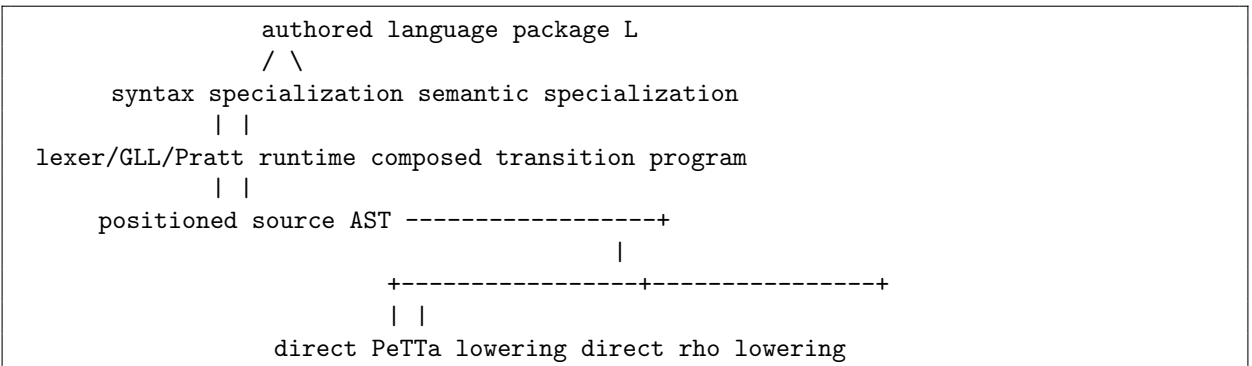
A GSLT separates a signature, an equational theory, and directed dynamics. The separation matters. An equation $t = t$ is harmless reflexivity in the equational center, but a directed rewrite $t \rightarrow t$ is a nonproductive computation step. A target syntax that uses one surface form for equations and executable rules cannot be allowed to erase this distinction during lowering.

Definition 1 (Scoped GSLT lowering). *A lowering from source fragment S to target presentation T consists of a term representation $\llbracket - \rrbracket$, realizations of the source signature and rules, and an observation map. On its declared domain it must:*

- (i) *preserve source equations;*
- (ii) *realize every source rewrite by target dynamics;*
- (iii) *reflect image behavior, so the target cannot invent a source step;*
- (iv) *preserve the declared observations; and*
- (v) *commute with the authored compositions for which it claims support.*

The qualification is important: no universal intermediate calculus is postulated. A parser automaton, a small execution instruction language, a PeTTa fragment, and a ρ process presentation may each be honest target GSLTs. A useful intermediate is admitted when it makes two actual lowerings simpler and its semantics is explicit. It is not declared universal merely because one prototype already emits it.

The intended diagram is



Composition preservation is ultimately a theorem, not a formatting property. The executable implementation currently provides deterministic source inventories, per-rule realization provenance, mutation gates, and severance tests. These are necessary integration evidence while the full theorem is instantiated, but they are not substituted for it.

3 Parsing and live syntax

Lexing and parsing form a separate specialization lane. A generic guarded lexer and generalized parser produce an ordered, normalized, source-positioned occurrence stream. Deterministic grammars may use generated cursor or Pratt fast paths; refusal by an accelerator returns authority to the generic parser rather than weakening the grammar.

MM0 makes the live nature of syntax unavoidable. A document can introduce term constructors and notation which later mathematical expressions use. Accordingly the runtime fold evolves an environment

$$\Gamma_0 \xrightarrow{d_1} \Gamma_1 \xrightarrow{d_2} \dots \xrightarrow{d_n} \Gamma_n,$$

and each mathematical field is lexed and parsed against the environment at its source position. This is a meta-GSLT transition: syntax is data whose meaning is an update to a parser state. It is not a reason to regenerate the target verifier.

The generic parser represents scalar sequences as ordinary lists. The MM0 secondary lexer uses its own scalar-list constructors. Their boundary is an authored codec, not an implicit C cast or a second handwritten decoder. This small edge illustrates the general rule: different GSLT presentations compose through named relations whose positive and negative behavior is testable.

4 Direct PeTTa specialization

PeTTa is the first didactic target because the residual program can remain close to the source semantics: equations, pattern matches, recursion, constructor-valued state, and result terms. The primary runner is CeTTa's `--lang petta`; the SWI-hosted implementation is a useful independent dialect check, not the authoritative deployment target.

Each authored relation receives a direct target definition. Binding-mode analysis records which arguments are ground at a call, which become ground on success, and whether the selected mode is relational or single-valued. Deterministic calls are compiled as deterministic target code; relational search remains relational. This avoids both extremes: running a deterministic checker through an unnecessary search engine, and destroying genuine nondeterminism with an accidental `once`.

4.1 Source constructors are not target operations

A subtle phase error appears when a source constructor has the same spelling as a PeTTa operation. A parsed AST may contain `(cons h t)` as inert source data. But a PeTTa equation head is an evaluation position: placing that form directly in the head can execute `cons` before the source rule is selected.

The renderer therefore classifies target-effectful heads. If a ground input pattern contains such a head, the equation receives a fresh slot and performs the authored structural match in its body:

```
; unsafe staging
(= (f (cons $h $t)) body)
```

```

; phase-correct staging
(= (f $__input_0)
  (let (cons $h $t) $__input_0 body))

```

Safe patterns remain in equation heads so recursive clause discrimination is not flattened. A generic canary uses a source constructor literally named `cons`; recursive ancestor canaries ensure that the repair does not turn all input patterns into catch-all clauses.

The current classifier is a target-specific table in the PeTTa renderer. A cleaner endpoint is to derive it from an authored PeTTa target presentation. This is future refactoring, not permission for source-language special cases in the renderer.

5 Direct ρ specialization

The ρ route consumes authored equations and rewrites directly. It does not translate the PeTTa schedule and does not factor through Horn clauses. Strict ρ supplies names, sends, receives, parallel composition, quotation, drop, and structural congruence. A small rhometta layer may be used when a target operation is explicitly presented and its reduction boundary is known.

Set-automaton matching provides a principled compilation strategy for a closed left-linear rewrite family [1]. Target channels are computed from the whole pattern family, and persistent receives realize rules. Nonlinear heads and decision premises must be linearized or presented as explicit target computations; they must not be hidden in an adapter.

Metamath checking itself is mostly deterministic and sequential. This does not make a ρ lowering useless. Rather, it supplies the principal soundness criterion: the image of a deterministic source checker must be determinate up to structural congruence. Milner’s encoding of functions as processes supplies the classical precedent [6].

Proof obligation 1 (Determinate image). *For every source state in the deterministic MM/MM0 checking fragment, any two one-step target reductions from its image are equivalent at the declared observation boundary. Every image reduction reflects a source transition. Consequently target scheduling cannot change verdict, position, witness, or history.*

The generic determinacy-to-confluence and sequential-realization schema is formalized. Its full MM and MM0 instantiations require the complete authored semantics and executable lowering witnesses.

6 Why Horn is not the canonical middle

Horn clauses can encode ordered checking if positions, prefixes, and schedules are reified as data. The question is not expressibility; sufficiently rich logic programming can express the computation. The question is whether the encoding preserves or obscures the native structure.

A set of ground facts erases order and multiplicity. Metamath database acceptance distinguishes both: declarations are processed in order, and a duplicate declaration can change acceptance. Therefore no pointwise statement-to-fact projection followed by positive Horn saturation can be a faithful account of the database verdict. A faithful Horn encoding must put the missing sequence back into its facts or control state. It relocates the checker rather than simplifying it.

For PeTTa the extra layer is especially unhelpful: PeTTa already lowers its own equations to a Prolog implementation. Emitting direct PeTTa lets its normal implementation choose indexing

and WAM-level execution. Reifying a second Horn program and interpreting it inside PeTTa duplicates the logic programming layer and moves source order into an auxiliary schedule.

Horn projections remain legitimate independent oracles for small relational canaries. They are not compiler authority, not a required backend, and not a performance architecture.

7 Metamath semantic shape

The natural Metamath verifier is an ordered, proof-producing transition system. Its state includes declarations, scoped hypotheses, symbol classes, disjoint-variable information, assertion frames, and source positions. A normal proof is a deterministic stack program. A compressed proof adds a dictionary and saved-result discipline but remains a deterministic check of a finished certificate.

The authored semantic package distinguishes:

- source folding and scope transitions;
- state-table lifetimes and ordered actions;
- normal and compressed proof decoding;
- assertion application and substitution;
- disjoint-variable obligations;
- total outcomes, positions, witnesses, and history.

The direct target work has executable PeTTa and ρ slices across the state action families and proof streams. Earlier source-owned native C machinery also demonstrates full-database parsing and checking performance, but it is not evidence that the direct PeTTa/ ρ compiler is complete. The two must not be conflated during integration.

8 MM0 as the second guest

MM0 is not a dialect flag on the Metamath checker. It is a second guest whose two stages must remain distinct [2]:

1. textual `.mm0` elaboration builds and validates the specification;
2. binary `.mmb` verification checks the encoded proof object against that specification.

The textual pipeline currently reaches assertion elaboration:

```
bytes -> generic guarded parser -> ordered AST
      -> sorts -> terms -> notation -> dynamic math parser
      -> expression elaboration -> axioms/theorems
```

Assertions use the live term and notation environment. Typed binders precede hypotheses; hypotheses and conclusions are elaborated at their source position; namespace collisions and sort mismatches produce explicit refusal.

Still required are definition declarations, textual proof execution, total outcomes for the complete specification, binary decoding, and a correspondence statement between the textual and binary stages. MM0 is the decisive guest-genericity test: these features must be added as authored capabilities, not as `if MM0` branches in either target renderer.

9 Correctness structure

Correctness is divided into construction laws and qualification evidence.

Construction laws. The desired theorem family states signature realization, equation preservation, rewrite realization, reflection of image steps, observation preservation, and compatibility with composition. Proofs live over GSLT objects and target presentations. No test corpus can replace these laws.

Qualification evidence. Deterministic regeneration, source digests, rule inventories, positive and negative canaries, source/target mutation, drift rejection, and severance make the implementation auditable. Removing a load-bearing authored rule must change the generated artifact and eliminate the corresponding behavior; C or another runtime must not rescue it.

Independent differentials. The Metamath route retains an independent Lean-verified checker differential. Agreement is powerful evidence about implementation, but the target compiler is not defined as “whatever happens to agree with the checker.” This avoids translation validation becoming the architecture.

10 Performance follows representation

The didactic target should be readable, but it need not be deliberately slow. The first performance law is to keep data as data. Runtime documents are streamed into state; large ground relations are stored once and indexed rather than emitted repeatedly as clauses. The second law is to derive control from semantic properties:

- deterministic ordered transitions become deterministic code;
- ground functional relations use direct lookup or native indexes;
- genuine relations preserve occurrence multiplicity and branching;
- tabling or fixpoints require an authored recursive relation;
- ρ concurrency is used only where source independence permits it.

Performance failures are classified before the goal is narrowed. A compiler that emits four projections of one runtime stream as four clause families has a representation defect, not evidence that the target runtime is inherently slow. Only after the input boundary and carrier are correct should target machine optimization begin.

11 Qualification snapshot

Table 1 separates current executable evidence from unfinished work. “Green” means the named bounded gates passed in the integration lineage; it does not mean the full project is complete.

The MM0 assertion gate currently checks five independent c.Petta outcomes and two direct ρ observations. It regenerates 436 PeTTa source-rule realizations and 415 source plus 176 target rules for the ρ artifact. Neither artifact contains a concrete .mm0 path or Horn/SLD/Datalog execution vocabulary.

Component	Evidence	Open boundary
Direct $\text{GSLT} \rightarrow \text{PeTTa}$ renderer	Source identity, per-rule provenance, binding modes, recursive canaries, source-constructor phase canary, mutation and severance green	Derive the target-effectful vocabulary from the PeTTa presentation; complete the composition theorem
Direct $\text{GSLT} \rightarrow \rho$ renderer	Strict-core target package, recursive/state/proof unit families and target identity gates green	Full MM/MM0 realization witnesses and full-corpus execution
Metamath	All authored state action kinds lowered and stepped; normal/compressed unit proof streams and total unit outcomes green	Full direct-target corpus acceptance and remaining scale work
MM0 textual syntax	Generic parser, live syntax fold, sorts, terms, notation, dynamic math parser, expressions, and assertions green in PeTTa and ρ	Definitions and textual proof execution
MM0 binary	Typed proof-machine sources preserved for reuse	Complete <code>.mmb</code> transition presentation and textual/binary correspondence
Formal correspondence	Generic composition, observation, certified planning, and determinate-image theory available	Executable compiler instances for the complete guests and targets

Table 1: Integration status as of 23 August 2026.

12 Integration discipline

The implementation spans a runtime, compiler tools, authored language packages, generated artifacts, and proofs. A merge should preserve that dependency order:

1. runtime correctness and target semantics;
2. generic parser and composition infrastructure;
3. direct PeTTa and ρ renderers;
4. authored Metamath packages and gates;
5. authored MM0 packages and gates;
6. generated artifacts regenerated from the merged tools;
7. performance changes, each retaining the full correctness surface.

Generated files are never used to adjudicate conflicts with their sources. They are regenerated after source/tool integration. Experimental oracles are kept outside authoritative aggregate gates. A branch that contains a useful optimization and an obsolete semantic path is not merged wholesale: the optimization is transplanted with its direct correctness canary.

13 Remaining research programme

The next substantial steps are:

1. author MM0 definition transitions with positive and negative examples;
2. complete textual MM0 proof execution and total outcomes;
3. derive the `.mmb` verifier from the typed binary machine and prove its intended relationship to textual elaboration;
4. instantiate the composition and observation-preservation theorem family for both direct targets;
5. run complete bounded corpora, then the large Metamath and MM0 databases;
6. optimize c.Petta and the ρ target only after profiles identify target-machine costs rather than representation defects;
7. add Prime as a target when its typing materially strengthens the construction, and derive a native C backend from the same authored semantics later.

The intended endpoint is not one universal execution machine. It is a family of explicit, compositional interpretations of one authored source package, each exploiting the target honestly and each preserving the observations that make a verifier meaningful.

14 Conclusion

The simplest description is also the most useful: write the language once, compile its semantics once per target, and run many documents through the result. Metamath and MM0 are attractive didactic guests because their checkers are small enough to read and strict enough to expose phase errors. PeTTa keeps the residual close to equations and recursion; ρ tests that the compiler does not rely on a guest-shaped schedule. Correctness resides in the compositional interpretation, while regeneration, mutation, severance, and independent differentials keep the executable implementation honest.

References

- [1] R. Bouwman and J. Erkens. Term rewriting based on set automaton matching. *Logical Methods in Computer Science*, 18(4), 2022.
- [2] M. Carneiro. Metamath Zero: The cartesian theorem prover. In *Proceedings of CPP*, 2020.
- [3] Y. Futamura. Partial evaluation of computation process—an approach to a compiler-compiler. *Systems, Computers, Controls*, 2(5), 1971.
- [4] N. D. Jones, C. K. Gomard, and P. Sestoft. *Partial Evaluation and Automatic Program Generation*. Prentice Hall, 1993.
- [5] L. G. Meredith and M. Radestock. A reflective higher-order calculus. *Electronic Notes in Theoretical Computer Science*, 141(5):49–67, 2005.
- [6] R. Milner. Functions as processes. INRIA Research Report 1154, 1990.
- [7] J. C. Reynolds. Definitional interpreters for higher-order programming languages. In *Proceedings of the ACM Annual Conference*, 1972.
- [8] D. H. D. Warren. An abstract Prolog instruction set. Technical Report 309, SRI International, 1983.